

CS683 — ADVANCED COMPUTER ARCHITECTURE

Programming Assignment 1: Task 2

Amey Dhole
26M0795

Parthsinh Thakor
26M0796

Deva Babu
26M0797

Sumit Chavan
26M0799

0.1 Experimental setup

Hardware setup

Component	Configuration
CPU	13th Gen Intel Core i5-13420H, topology is 4 P-cores + 4 E-cores
L1d cache	48 KiB per P-core
L2 cache	1.25 MiB per P-core
L3 cache	12 MiB
Memory	16 GiB
Vector ISA	SSE, AVX, AVX2, FMA — no AVX-512

Software setup

We isolated a P core using kernel boot parameters and then pinned the output binary to that core for taking measurements.

2 Task 2: Matrix multiplication (SGEMM)

2.1 Task 2A: Software prefetching

Data deliverables

Prefetch distance	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
16	14.50×	11.69×	14.19×	16.26×
32	14.90×	13.07×	14.95×	16.88×
48	14.91×	9.69×	15.23×	16.74×
64	13.90×	10.80×	14.99×	16.31×
128	14.09×	12.36×	15.45×	17.14×

Table: Prefetch distance analysis over different prefetch distances

Metric	Dist.	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
L1-D	16	7,060,629	59,816,403	500,550,589	4,015,728,291
	32	6,610,880	52,234,091	421,555,719	3,413,171,798
	48	6,779,197	52,102,216	410,349,514	3,306,465,936
	64	6,986,483	52,877,440	412,596,787	3,307,128,406
	128	7,941,259	56,803,000	428,251,527	3,344,326,868
L2	16	19,667	710,035	13,113,774	1,154,316,436
	32	15,396	605,349	13,942,918	1,040,692,607
	48	17,688	2,389,510	13,769,628	1,040,696,621
	64	13,049	3,006,667	13,321,035	1,076,439,949
	128	18,531	810,266	13,566,582	1,036,432,225
LLC	16	13,199	124,429	2,813,650	1,006,952,175
	32	9,599	107,304	3,749,157	966,505,870

Metric	Dist.	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
	48	11,933	141,726	3,418,993	980,164,698
	64	6,440	173,945	3,021,396	1,017,123,942
	128	13,269	135,338	3,212,210	976,617,086

Table: Prefetch parameter analysis using different cpu metrics

Analysis of Hardware prefetcher being turned on:

The absolute cache misses has dropped significantly across all levels of cache hierarchy when Hardware prefetcher is turned on. Especially as the matrix sizes grow in size the cache misses are dropped at very significant amount, at matrix size of 2048 we get 78x less cache misses, this behaviour happens because the hardware stream prefetcher requires a run of sequential access and with huge matrix sizes that becomes more convenient and hence the misses are dropped significantly.

Another interesting thing is that as the matrix sizes increases the speedup decreases as compared to the speedup achieved with hardware prefetcher off. This behaviour happens because by turning hardware prefetcher on, now the naive code also gets optimised which in turn affects the speedup of our prefetch code as the speedup is relative measurement.

HW prefetcher	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
On	16.69×	14.75×	14.07×	12.57×
Off	14.90×	13.07×	14.95×	16.88×

Table: Effects of hardware prefetchers on software prefetching

Analysis deliverables

q1: The speed up generally increases with increase in matrix sizes(as shown in the table below)

q2: On average the optimal prefetch distance is 32. This prefetch distance provides overall better performance with different sizes of matrices. It is also a optimal choice based on the Cache miss pattern at multiple levels of care hierarchy

q3: There is a significant performance improvement when we use T0 as compared to T1 or T2

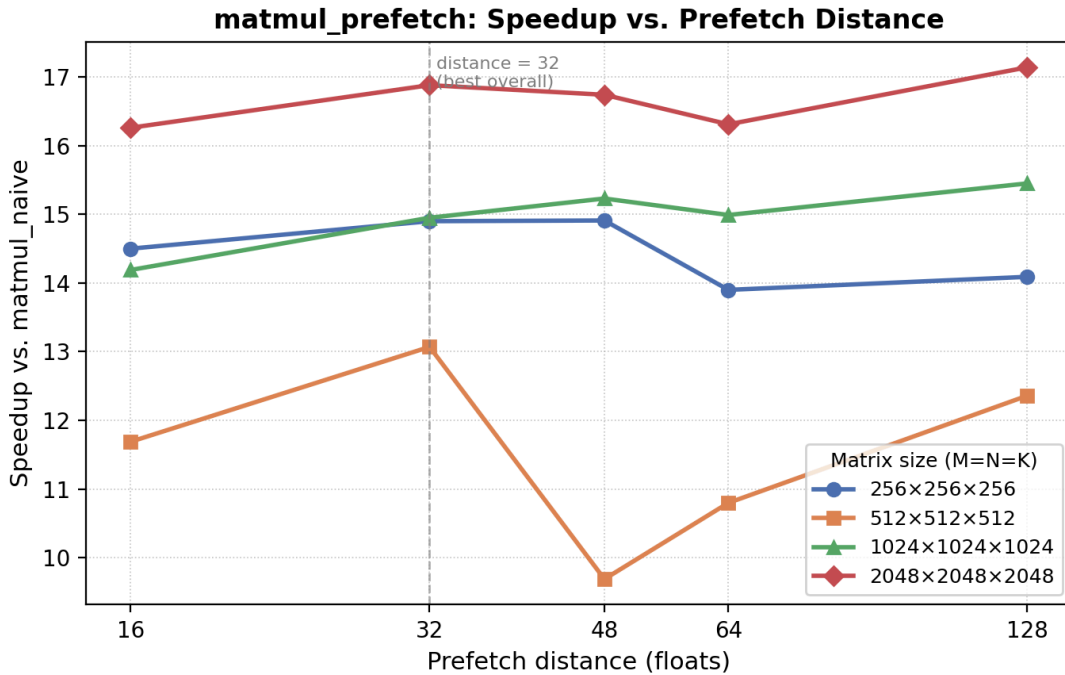


Figure 1: Speedup of `matmul_prefetch` over `matmul_naive` versus prefetch distance, for each matrix size. Distance 32 is the best overall compromise across sizes.

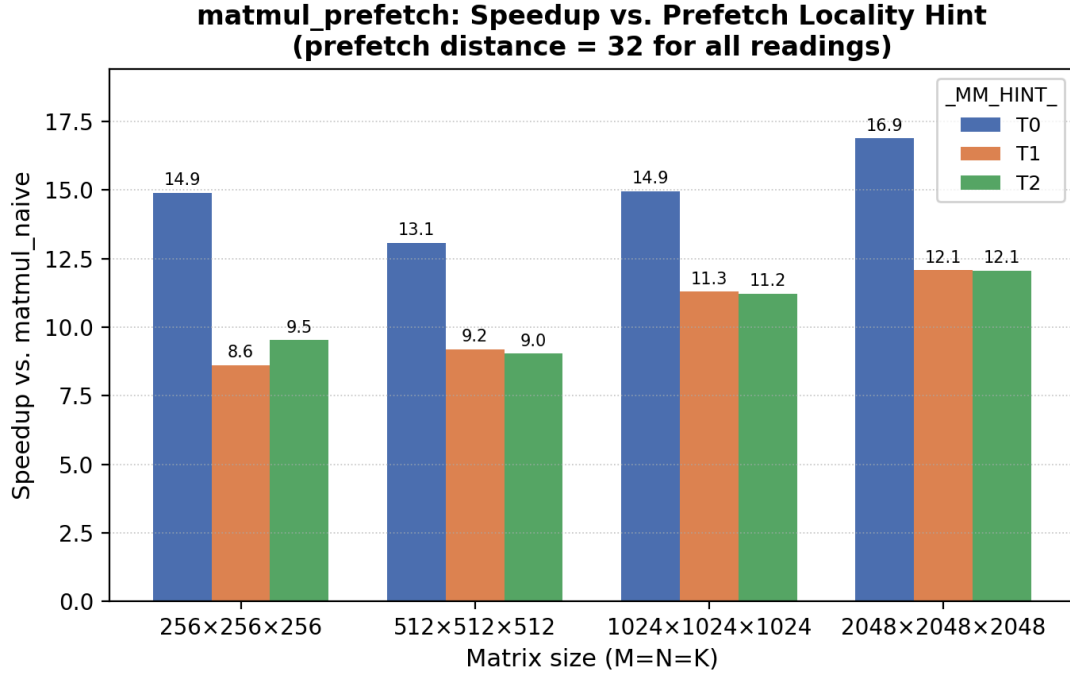


Figure 2: Speedup versus the prefetch locality hint (`_MM_HINT_T0` / `T1` / `T2`) at a fixed prefetch distance of 32. `T0` wins at every matrix size.

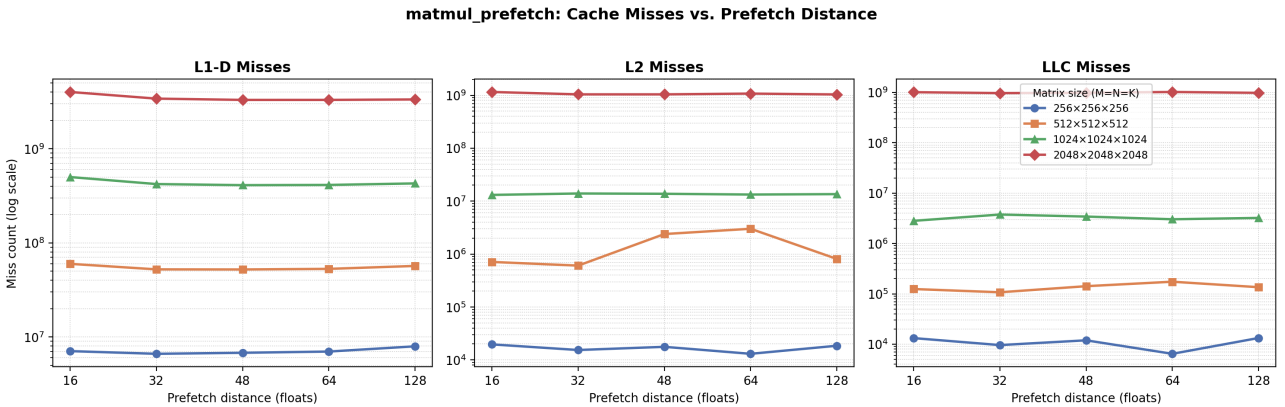


Figure 3: L1-D, L2 and LLC miss counts (log scale) versus prefetch distance, for each matrix size.

2.2 Task 2B: SIMD (Single Instructor Multiple Deadlines)

Analysis Deliverables

- q1. Doubling SIMD width improves performance for every matrix size.
- q2. Speedup gradually falls as matrix grows.
- q3. Instruction count did not fall when doing SIMD, we have reasoned that it is because we were profiling the binary which runs the naive as well as SIMD.
- q4. Execution Time ratio steeply falls as the matrix sizes.

Deliverables

q1: SIMD implementation substantially outperforms the non-SIMD implementation over all the tested matrix sizes. Also as the matrix sizes increases the speedup decreases for both 128 bit SIMD as well as for 256 bit SIMD, this is because as the matrix size grows over most effective cache capacity, cache misses and memory bandwidth increasingly limit performance. For every matrix size, the 256 bit SIMD outperforms the 128 bit SIMD. These result is quite expected as 256-bit register can process twice as many values as compared to 128-bit register.

q2: 256-bit SIMD gave the maximum speed up.

Matrix Size	SIMD				Without SIMD	
	SIMD width 128		SIMD width 256			
	Instruc-tions (M)	Execution Time	Instruc-tions (M)	Execution Time	Instruc-tions (M)	Execution Time
256	659.68	0.765 ms	645.32	0.478 ms	620.38	8.07 ms
512	5,194.07	7.714 ms	5,057.22	5.724 ms	4,888.86	55.58 ms
1024	41,170.41	76.117 ms	40,104.55	59.397 ms	38,886.06	487.66 ms
2048	328,261.32	1291.929 ms	320,231.07	1103.235 ms	310,339.38	4838.61 ms

Table: Matrix Sizes vs Instruction size & Execution time for SIMD and non SIMD

2.3 Task 2C: Software prefetching + SIMD

Optimised version gives better performance across all matrix sizes. SIMD implementation is the only one which degrades in performance as the matrix size increases, this is because there are more cache misses. Both Prefetch and SIMD + Prefetch shows increase in performance as Matrix size increases.

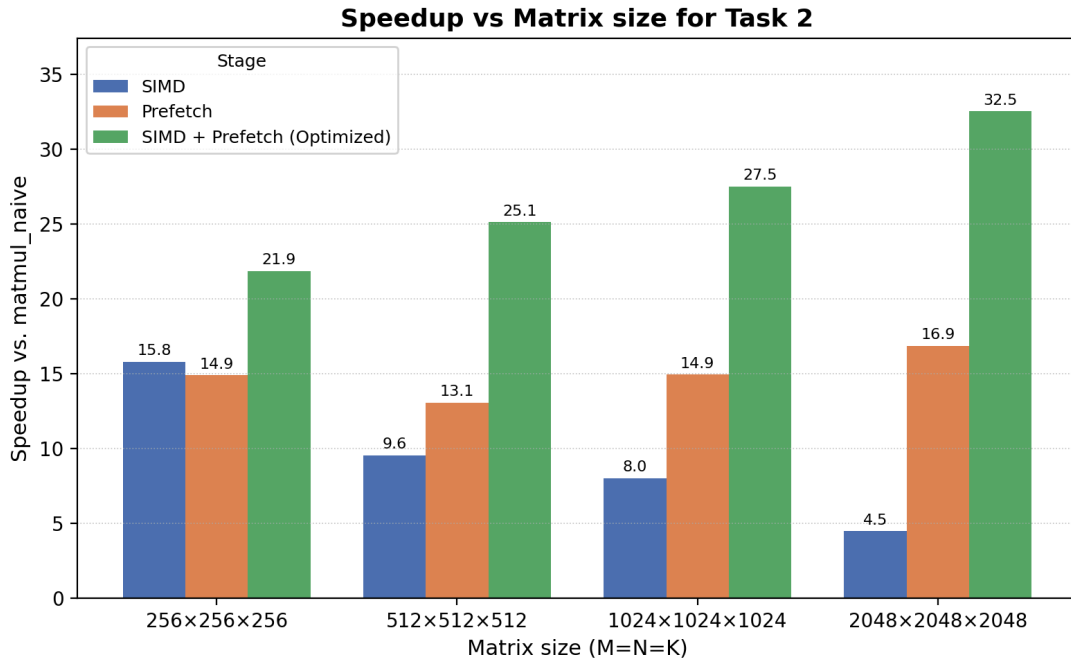


Figure 4: Matrix multiplication speedup of different techniques over naive version